

Fundamentos de Generación de Código

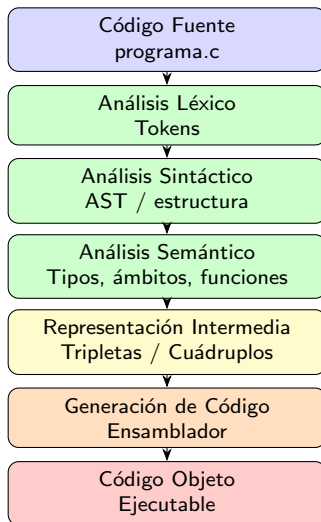
Compiladores

Objetivo de la Clase

Al finalizar la clase, el estudiante comprenderá:

- el pipeline general de un compilador;
- qué hace la fase de generación de código;
- qué son AST, tripletas y cuádruplos;
- cómo se genera ensamblador;
- cómo se traducen expresiones y llamadas a función.

Pipeline General del Compilador



¿Qué es Generación de Código?

Definición

La generación de código transforma estructuras abstractas validadas por el compilador en instrucciones cercanas a la máquina.

La entrada del generador puede ser:

- AST,
- tripletas,
- cuádruplos,
- código de tres direcciones.

La salida puede ser:

- ensamblador,
- código objeto,
- ejecutables.

Ejemplo Base

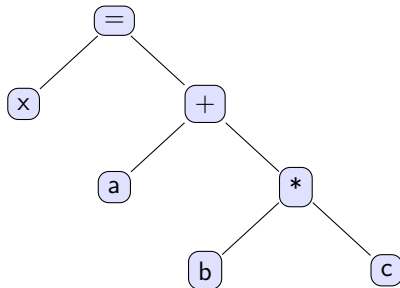
Usaremos:

```
x = a + b * c;
```

Este ejemplo permite estudiar:

- precedencia;
- AST;
- tripletas;
- cuádruplos;
- generación de ensamblador.

AST de la Expresión



Interpretación

Primero se calcula $b * c$, después se suma con a .

Código de Tres Direcciones

Para:

```
x = a + b * c;
```

se genera:

```
t1 = b * c  
t2 = a + t1  
x  = t2
```

Cada instrucción contiene una sola operación principal.

Tripletas

Forma general:

$$(op, arg_1, arg_2)$$

Para:

$$x = a + b * c$$

Índice	Operador	Arg1	Arg2
0	*	b	c
1	+	a	(0)
2	=	x	(1)

Idea

Las tripletas referencian resultados previos usando índices.

Cuádruplos

Forma general:

$(op, arg_1, arg_2, resultado)$

Índice	Operador	Arg1	Arg2	Resultado
0	*	b	c	t1
1	+	a	t1	t2
2	=	t2	-	x

Ventaja

Los cuádruplos hacen explícitos los temporales.

Ejemplo: Tripleta a Ensamblador

Tripleta:

$(*, b, c)$

Interpretación:

$resultado_0 = b * c$

Ensamblador conceptual:

```
movl b, %eax  
imull c, %eax
```

- b se carga en %eax.
- c multiplica el contenido del registro.
- El resultado queda en %eax.

Ejemplo Completo: Tripletas a ASM

Tripletas:

```
0: (*, b, c)
1: (+, a, (0))
2: (=, x, (1))
```

Ensamblador conceptual:

```
movl b, %eax
imull c, %eax

movl a, %ebx
addl %eax, %ebx

movl %ebx, x
```

La tripleta 0 produce el valor temporal utilizado por la tripleta 1.

Ejemplo: Cuádruplos a ASM

Cuádruplos:

```
(*, b, c, t1)
(+, a, t1, t2)
(=, t2, --, x)
```

Ensamblador conceptual:

```
movl b, %eax
imull c, %eax
movl %eax, t1

movl a, %ebx
addl t1, %ebx
movl %ebx, t2

movl t2, x
```

Observación

Los cuádruplos generan ensamblador más explícito porque los temporales

Ejemplo: Llamada a Función

Código C:

```
r = suma(a, b);
```

Código intermedio:

```
param a  
param b  
t1 = call suma, 2  
r = t1
```

Llamada a Función a Ensamblador

Ensamblador conceptual:

```
movl a, %edi  
movl b, %esi  
  
call suma  
  
movl %eax, r
```

- Los argumentos se colocan en registros.
- `call` transfiere el control.
- El valor de retorno queda en `%eax`.

Bloques Básicos

Un bloque básico es una secuencia de instrucciones con:

- una sola entrada;
- una sola salida;
- sin saltos internos.

Ejemplo:

$$t1 = b * c$$

$$t2 = a + t1$$

$$x = t2$$

Demostración Terminal

Generar ensamblador:

```
gcc -S programa.c -o programa.s
```

Mostrar ensamblador:

```
cat programa.s
```

Generar objeto:

```
gcc -c programa.s -o programa.o
```

Generar ejecutable:

```
gcc programa.o -o programa
```


Resumen Conceptual

- El análisis léxico produce tokens.
- El análisis sintáctico produce estructura.
- El análisis semántico valida significado.
- La generación de código produce instrucciones ejecutables.
- Las tripletas usan índices.
- Los cuádruplos usan temporales explícitos.
- El ensamblador utiliza registros e instrucciones reales.

Frase Final

Idea Central

La generación de código transforma estructuras abstractas validadas por el compilador en instrucciones ejecutables para la arquitectura destino.

Código Fuente $\rightarrow$ AST $\rightarrow$ Tripletas / Cuádruplos $\rightarrow$
ASM $\rightarrow$ Ejecutable